Day 24 — Gather & Join (Hands-On Patterns)

1. Recap — What Join and Gather Each Do

Step	Definition from Class
Join	Used to **combine multiple branches into a single branch** — **it doesn't do anything with the messages** themselves.
Gather	Used to **combine different messages into a single message.**

2. Demo ① — Parallel Multicast with Join + Gather

Setup: Postman → iFlow; body = **Employee details (raw → XML)**.

<table>
<tr><td rowspan="2">Sender</td><td rowspan="2">HTTPS →
Start Message</td><td rowspan="2">Parallel Multicast</td><td>Branch 1 → CM1</td><td rowspan="2">→ Join → Gather → End Message</td><td rowspan="2">Receiver</td></tr>
<tr><td>Branch 2 → CM2</td></tr>
</table>

Step	Message Body Set in Content Modifier
CM1	`<root> ${body} <route>Branch 1</route> </root>`
CM2	`<root> ${body} <route>Branch2</route> </root>`

Each branch wraps the original body (`${body}`) and adds its own marker, so after Join + Gather you can **see which branch produced which part** of the combined message.

The Combined Output — "Multimap" Format

When Gather uses the **Combine** algorithm, the messages are put together in **multimapping format** — an **SAP-developed format** (it comes from SAP's Process Integration "split and merge" mapping). Illustrative shape of the result:

```
<multimap:Messages xmlns:multimap="http://sap.com/xi/XI/SplitAndMerge">
  <multimap:Message1> <root> ...employee... <route>Branch 1</route> </root> </multimap:Message1>
  <multimap:Message2> <root> ...employee... <route>Branch2</route> </root> </multimap:Message2>
</multimap:Messages>
```

> **Tag names are case-sensitive:** your sketch writes `<multimap:message1>` in lowercase. In SAP's format the wrapper is `multimap:Messages` and the parts are `multimap:Message1` , `multimap:Message2` ... with capital **M**. Check the exact output of your own iFlow in **Trace**, since a downstream mapping must use the exact tag names.

3. What Are "Different Messages" for Gather?

> **Gather works within one run:** an iFlow is triggered → it gets **one Message ID** → within this **same Message ID**, if you used a **Splitter** (or Multicast), the incoming payload is divided into several messages. **To combine those messages again into one payload, we use Gather.**

Messages Gather CAN combine	Messages Gather CANNOT combine
Messages created **inside CPI** from **one** incoming message (Splitter splits, Multicast copies) — same Message ID	Separate messages arriving from the **source** as **different runs with different Message IDs**

For the second case you would use the **Aggregator** (Day 23): it collects separate incoming messages across runs until a completion condition is met.

4. Demo ② — Splitter + Gather

Sender	HTTPS →	Start Message	→	General Splitter	→	Gather	→	End Message	SFTP →	Receiver

What is the use of Splitter + Gather here? In this scenario, no use — splitting and then immediately gathering gives you back what you started with. **But if there is any functional step in between** (some processing per split message), then the splits are handled one by one and **should be combined afterwards — that is where we use Gather.**

Real-Time Scenario — Northwind OData API

Requirement: the source sends **2 employee records**. For those 2 employees, fetch data from the **Northwind API** (an open-source OData demo service), **combine** the results, and send them back as **one single response**.

Assumption: in a single API call we can get **only one** record's data.

Sender	HTTPS →	Start Message	→	GS (General Splitter)	→	CM	→	RR (Request Reply)	→	Gather	→	End Message → SFTP

RR — **OData** →	**Northwind Receiver**

Step	What It Does
General Splitter	Splits the 2-employee input into Emp ① and Emp ② messages
Content Modifier	Prepares the call for each employee (e.g., puts that employee's ID into a header used in the OData request)
Request Reply (OData)	Calls the Northwind API once per employee and gets that employee's data
Gather	Combines the individual responses into **one single message**

> **Why no Join here:** a Splitter keeps everything on **one path**, so Gather alone is enough. Join is only needed when the messages arrive on **several branches** (section 5).

5. Demo ③ — Same Requirement, but with Multiple Branches

Now assume the data is coming from **multiple branches**:

Sender	HTTPS → Start	Parallel Multicast	Branch 1 (Emp ①): CM → RR	→ **Join** → **Gather** →	End → SFTP
			Branch 2 (Emp ②): CM → RR		

Each RR — **OData** →	Northwind Receiver

> **Here, Gather doesn't work when you are giving it multiple inputs.** To overcome this, **we use Join** — **Join combines multiple branches into a single branch**, and Gather then takes that single path.
>
> **Rule:** whenever you are using a **Multicast**, you should use **Join and Gather together.**

> **Remember from Days 21 and 23:** in a Join + Gather setup, if one incoming branch fails, the processing stops (unlike a plain Parallel Multicast, where the other branches would continue).

6. How Does Gather Know What It Is Combining? — Aggregation Strategy

Question: how will Gather combine? How will it know what type of message it is receiving? → **Gather** → **Aggregation Strategy** has two settings: **Incoming Format** and **Aggregation Algorithm**.

① Incoming Format (4 Options)

Any · Plain Text · XML (Different Format) · XML (Same Format) — and note that **CPI mainly works with XML**.

② Aggregation Algorithm — Which Options Go with Which Format

Incoming Format	Aggregation Algorithms Available
XML (Same Format)	a. **Combine** · b. **Combine at XPath** · c. **Tar** · d. **Zip**
XML (Different Format)	a. **Combine** · b. **Tar** · c. **Zip**
Plain Text	a. **Tar** · b. **Zip** · c. **Concatenate** (text data)
Any	a. **Tar** · b. **Zip**

Verified exactly against SAP's documentation — every combination in your table is right.

What Each Algorithm Does

Algorithm	Behaviour
Combine	Combines the incoming XML messages without any condition, in the **multimap** format (section 2)
Combine at XPath	Combines the messages **under a node you specify** (*Combine at target (XPath)*) — for same-format XML only
Concatenate	Plain text only: joins the texts one after another — e.g., `abc` + `efg` → `abcefg`
Zip / Tar	Packs the messages into an **archive**; works with every incoming format

Details worth knowing:

- **Combine at XPath fails with an exception** if the XPath doesn't match an element in any branch — so double-check the path (and namespaces, Day 20).
- **Zip/Tar need a unique file name per message:** the **File Name** field takes an expression (e.g., `${header.id}.xml` — the header must be set *before* the Gather). Duplicate names inside one archive cause an error.
- **Correction to my Day 23 PDF:** I listed only three algorithms (Combine, Combine at XPath, Concatenate). Your Day 24 table is the complete one — Zip and Tar exist too.

7. Recap — General Splitter vs. Iterating Splitter

General Splitter	Iterating Splitter
Returns the entire payload structure — it **preserves the encapsulating nodes/elements** with each split message	**Does not preserve** the encapsulating elements or nodes — only the split node

(Same distinction as Day 22 — it's a standard interview question, and your trainer flagged "multimap messaging" as important alongside it.)

8. Quick Interview Answers

Question	Short Answer
What does Join do?	Merges multiple branches into one path without touching the messages
What does Gather do?	Combines multiple messages into one message
Which messages can Gather combine?	Messages created within the same run (same Message ID) by a Splitter or Multicast
Splitter + Gather — Join needed?	No — one path. Multicast + Gather — Join is required first
Gather's two settings?	Incoming Format and Aggregation Algorithm
Algorithms for XML (Same Format)?	Combine, Combine at XPath, Tar, Zip
Which algorithm joins plain text?	Concatenate
What is the multimap format?	SAP's split-and-merge XML wrapper (`multimap:Messages` / `Message1…`) produced by Combine
Real example of Splitter +	Split employees, call an OData API per employee with Request Reply, gather the responses into

Gather?	one message